

ALIANCA-LOG-ESCOPO-TECNICO-R01

created: 2026-06-21T20:00
updated: 2026-06-21T20:41
type: escopo-tecnico
projeto: ALIANCA-LOG
cliente: Rotta Logística
contato: Matheus Rotta
responsavel_uzzai: "Vitor Pirolli"
status: draft
versao: R01
tags:

- alianca-log
 - escopo
 - tecnico
 - rotta
 - logistica
- dg-publish: false

🔧 ALIANÇA LOG — Plano de Escopo Técnico R01

Sistema de Gestão de Entregas · Rotta Logística × UzzAI

Documento de referência para desenvolvimento. Define exatamente o que será construído, quem faz, com qual stack, em qual prazo e com quais critérios de aceite.

Partes: Rotta Logística (Matheus Rotta) × UzzAI (Vitor Pirolli / Luis Fernando Boff / Pedro Vitor Pagliarin)

📑 ÍNDICE

- 1. Contexto e Objetivo
- 2. Escopo do MVP A — O que está dentro
- 3. Escopo do MVP B — O que está dentro
- 4. Fora do Escopo (v2/v3)
- 5. Stack Técnico Definitivo
- 6. Arquitetura do Sistema
- 7. Banco de Dados — Schema
- 8. Telas e Fluxos por Perfil
- 9. Plano de Sprints e Timeline
- 10. Equipe e Papéis
- 11. Serviços UzzAI incluídos
- 12. Critérios de Aceite (DoD)
- 13. Contrato e Pagamento
- 14. Glossário

1. CONTEXTO E OBJETIVO

1.1 Quem é o Cliente

Rotta Logística — empresa de distribuição last-mile que opera em toda a Serra Gaúcha.

Dado Operacional	Valor
Entregas/mês	2.000+

Dado Operacional	Valor
Entregadores	16
Veículos	9
Clientes (embarcadores)	~20 empresas
Cobertura	Serra Gaúcha (Caxias, Gramado, Canela, Petrópolis...)
Sistema atual	Planilha + WhatsApp + canhoto físico

1.2 O Problema Principal

Toda a operação é manual. Quando um canhoto se perde, não há comprovante digital. Quando o cliente quer saber o status da entrega, liga ou manda mensagem. A gerência só sabe o que aconteceu quando o motorista volta ao escritório.

Prejuízo confirmado por falta de rastreabilidade. O inegociável:

"Controle dos canhotos em tempo real." — Matheus Rotta

1.3 Objetivo do Sistema

Construir o **Aliança Log**: plataforma web que digitaliza o ciclo completo do canhoto, conectando gerência, motoristas e clientes finais em tempo real.

1.4 Respostas Determinantes para o Escopo

As seguintes respostas do cliente definem decisões técnicas chave:

Pergunta	Resposta	Decisão Técnica
App nativo ou site?	Site, se baratear	PWA (Next.js) — sem app nas lojas
Sinal nas rotas?	Sim, sinal fraco na serra	Offline-first com Service Worker + IndexedDB
Como entregas entram?	Excel das empresas clientes	Parser de .xlsx no backend
Notificações automáticas?	Não precisa, portal basta	Sem push notifications / WhatsApp API
Quantos clientes com login?	~20 empresas	Multi-tenant read-only simples

2. ESCOPO DO MVP A

Tudo listado aqui **será entregue** dentro do prazo e valor da Proposta A.

2.1 Módulo de Autenticação

- ☐ Login por e-mail + senha (Supabase Auth)
- ☐ 3 roles distintos: `gerencia`, `motorista`, `cliente_final`
- ☐ Middleware Next.js protege rotas por role
- ☐ Redirecionamento automático pós-login para a tela correta do role
- ☐ Sessão persistente (sem re-login a cada acesso)
- ☐ Logout com limpeza de sessão
- ☐ Tela de recuperação de senha (e-mail automático Supabase)

2.2 Painel da Gerência

2.2.1 Dashboard Principal

- ☐ Visão de todas as entregas do dia com status em tempo real (WebSocket)
- ☐ Contadores no topo: Total / Aceitas / Recusadas / Retidas / Com Pendência / Em Aberto
- ☐ Lista de entregas com: NF, destinatário, motorista, empresa cliente, status, horário da última atualização

- ☐ Filtro por: status, motorista, empresa cliente
- ☐ Status atualiza automaticamente sem necessidade de recarregar a página

2.2.2 Visualização do Canhoto

- ☐ Ao clicar em uma entrega, abre modal com foto do canhoto em alta resolução
- ☐ Informações detalhadas: NF, status, motorista, horário, observação (item faltante)
- ☐ Se offline e ainda não sincronizado: exibe "Pendente sincronização"

2.2.3 Importação de Entregas via Excel

- ☐ Upload de arquivo .xlsx ou .csv
- ☐ Parser lê as colunas e exibe preview antes de confirmar
- ☐ Mapeamento configurável de colunas (ex.: "NF" pode ser "NOTA FISCAL" ou "NF_NUM" etc.)
- ☐ Atribui motorista a cada entrega (dropdown na tela de confirmação)
- ☐ Confirmação cria todas as entregas do dia no banco

Colunas obrigatórias no Excel:

- Número da NF
- Nome do destinatário
- Endereço de entrega
- Empresa cliente (embarcador)

Colunas opcionais:

- Motorista sugerido
- Observações

2.2.4 Gestão de Cadastros

- ☐ Cadastro de motoristas (nome, e-mail, telefone)
- ☐ Cadastro de empresas clientes (nome, e-mail de acesso)
- ☐ Criação de logins para motoristas e clientes
- ☐ Inativar/reactivar usuários

2.3 App do Motorista (Mobile-First)

Design — Princípios

O app do motorista precisa ser **ultra-simples**. O motorista usa com uma mão, ao sol, com pressa. Cada tela tem no máximo 1 ação principal.

2.3.1 Lista de Entregas do Dia

- ☐ Ao fazer login, exibe imediatamente as entregas atribuídas ao dia
- ☐ Cada card mostra: NF, nome do destinatário, endereço, status atual
- ☐ Cores indicadoras de status (verde = aceito, vermelho = recusado, amarelo = pendente)
- ☐ Ordenação por ordem de atribuição (gerência define a sequência na importação)
- ☐ Contador de progresso: "8 de 16 entregas concluídas"

2.3.2 Tela de Registro de Canhoto

Ao tocar em uma entrega, o motorista vê:

- ☐ Dados da entrega (NF, destinatário, endereço)
- ☐ Botão "TIRAR FOTO DO CANHOTO" (abre câmera do celular)
- ☐ Preview da foto tirada com opção de refazer
- ☐ Seleção de status — 4 botões grandes:
 - ☒ NOTA ACEITA
 - ☒ NOTA RECUSADA

-  CANHOTO RETIDO
 -  ITEM FALTANDO
- ☐ Se "ITEM FALTANDO": campo texto obrigatório "Qual item faltou?"
 - ☐ Botão "CONFIRMAR E ENVIAR"
 - ☐ Feedback visual de sucesso ("Registrado! ) ou "Salvado offline — enviando quando tiver sinal"

2.3.3 Funcionamento Offline

- ☐ Ao abrir o app sem internet, as entregas do dia carregam do cache local (IndexedDB)
- ☐ Motorista consegue tirar foto e registrar status mesmo sem sinal
- ☐ Foto e dados são salvos localmente com flag **sincronizado: false**
- ☐ Quando o sinal volta (mesmo 2G), sync automático em background
- ☐ Banner discreto na tela mostra: "3 registros aguardando sincronização"
- ☐ Após sync, banner desaparece e gerência vê os dados atualizados

2.3.4 Barra de Pesquisa

- ☐ Campo de busca na lista de entregas
- ☐ Busca por NF ou nome do destinatário
- ☐ Resultado em tempo real ao digitar

2.4 Portal do Cliente Final

O cliente (ex.: Leite Travizão, Aurora) acessa um portal **read-only** e vê apenas **suas próprias** entregas.

2.4.1 Lista de Notas

- ☐ Lista de todas as NFs atribuídas à empresa deste login
- ☐ Filtro por data (hoje, semana, mês)
- ☐ Filtro por status (todas, aceitas, pendentes, com problema)
- ☐ Ordenação por data, NF ou status

2.4.2 Detalhe da Entrega

- ☐ Status em tempo real (atualiza automaticamente via WebSocket)
- ☐ Foto do canhoto assim que enviada pelo motorista
- ☐ Horário do registro
- ☐ Motorista que realizou a entrega
- ☐ Observação (item faltante, se houver)

3. ESCOPO DO MVP B

Tudo listado aqui é **adicional ao MVP A** — entregue no prazo de 2–3 meses. O MVP B pressupõe que o MVP A está em produção e validado.

Valor: R\$ 25.000 – R\$ 45.000 (setup total, incluindo o A) · **Prazo:** 2–3 meses a partir do início

3.1 Módulo de Roteirização (Motorista)

3.1.1 Sugestão de Rota via GPS

O motorista recebe, além da lista de entregas, uma **sequência otimizada** de quais endereços visitar primeiro — minimizando distância e tempo.

- ☐ Integração com **Google Maps Directions API**
- ☐ Ao iniciar o dia, botão "Roteirizar Entregas" na lista do motorista
- ☐ Sistema calcula ordem otimizada com base nos endereços cadastrados
- ☐ Exibe sequência numerada: 1º, 2º, 3º... com endereço e NF de cada parada

- ☐ Botão "Abrir no Maps" para cada parada (abre Google Maps nativo do celular com navegação turn-by-turn)
- ☐ Roteirização manual: motorista pode reordenar arrastando os cards
- ☐ Estimativa de tempo total da rota

3.1.2 Restrições de Janela de Horário (opcional, se cliente precisar)

- ☐ Campo "janela de entrega" na importação do Excel (ex.: "apenas de manhã")
- ☐ Roteirização respeita a janela ao calcular a sequência

Custo adicional de infra: Google Maps Directions API — estimativa ~R\$ 50–150/mês dependendo do volume de rotas calculadas por dia.

3.2 Painel de NF por Empresa (Motorista)

O motorista ganha uma visão organizada por empresa cliente — útil quando o caminhão carrega entregas de várias empresas ao mesmo tempo.

- ☐ Nova aba no app: "Por Empresa"
- ☐ Agrupa as entregas do dia por empresa cliente (Leite Travizão / Aurora / etc.)
- ☐ Dentro de cada empresa: lista de NFs com status
- ☐ Barra de pesquisa global (busca em todas as empresas ao mesmo tempo)
- ☐ Filtros: status, empresa, bairro/cidade
- ☐ Badge com contagem pendente por empresa

3.3 KPIs de Desempenho dos Motoristas (Gerência)

A gerência ganha uma tela de acompanhamento individual de cada motorista.

3.3.1 Dashboard de Motoristas

- ☐ Lista de todos os motoristas com indicadores do mês corrente:
 - Total de entregas realizadas
 - Taxa de entrega bem-sucedida (ACEITA / total)
 - Taxa de problemas (RECUSADA + RETIDA + ITEM FALTANDO)
 - Tempo médio entre entregas (estimado pelo horário dos registros)
- ☐ Ordenação por qualquer KPI (ranking)
- ☐ Seletor de período: hoje / semana / mês / período customizado

3.3.2 Perfil Individual do Motorista

- ☐ Ao clicar em um motorista: histórico completo de entregas
- ☐ Gráfico de linhas: entregas/dia ao longo do período
- ☐ Top 5 ocorrências mais comuns desse motorista
- ☐ Botão "Exportar histórico" (PDF simples)

3.4 Controle Financeiro — Rentabilidade por Entrega (Gerência)

A gerência pode analisar quais rotas, clientes ou motoristas são mais lucrativos.

3.4.1 Cadastro de Custos

- ☐ Tela de configuração: custo por km (combustível + desgaste)
- ☐ Custo hora do motorista (valor configurável por motorista)
- ☐ Valor cobrado por entrega por empresa cliente (tarifa contratual)

3.4.2 Cálculo de Rentabilidade

- ☐ Por entrega: receita – (custo km × distância estimada) – (custo hora × tempo estimado)
- ☐ Por rota (conjunto de entregas do dia de um motorista)
- ☐ Por empresa cliente: soma de todas as entregas do período
- ☐ Indicador visual: ● Lucrativa / ● Neutra / ● Prejuízo (por entrega e por rota)

Nota técnica: Distância estimada via Google Maps Distance Matrix API. O cálculo exato depende dos dados de custo cadastrados — precisão aumenta com o tempo de uso.

3.4.3 Tela de Rentabilidade

- ☐ Visão consolidada: receita total do mês vs. custo total estimado
- ☐ Tabela por empresa cliente: receita, custo, margem
- ☐ Tabela por motorista: custo de operação vs. entregas realizadas
- ☐ Filtro por período

3.5 Gráficos e Dashboards por Cliente (Gerência)

A gerência e os clientes finais ganham dashboards mais ricos.

3.5.1 Dashboard Gerência — Visão Analítica

- ☐ Gráfico de barras: entregas por status por dia (últimos 30 dias)
- ☐ Gráfico de pizza: distribuição de ocorrências (aceitas / recusadas / retidas / pendências)
- ☐ Gráfico de linhas: evolução do volume de entregas ao longo do tempo
- ☐ Mapa de calor por cidade: onde concentram as entregas e os problemas
- ☐ Tabela: top 5 empresas clientes por volume

3.5.2 Dashboard por Empresa Cliente

- ☐ Ao selecionar uma empresa: visão consolidada só para ela
- ☐ Total de entregas no período
- ☐ Taxa de sucesso das entregas dela especificamente
- ☐ Histórico de ocorrências (RETIDAS, RECUSADAS, ITEM FALTANDO) com detalhes

3.6 Filtros Avançados — Portal do Cliente Final

O portal do cliente ganha mais poder de filtragem.

- ☐ Filtro por cidade / bairro de entrega
- ☐ Filtro por período customizado (calendário date picker)
- ☐ Filtro por status (múltipla seleção)
- ☐ Filtro por motorista que realizou a entrega
- ☐ Ordenação por qualquer coluna da tabela
- ☐ Exportação do filtro atual como planilha Excel (CSV simples)

3.7 Relatórios Exportáveis (Gerência)

- ☐ Relatório de entregas do período: filtros → exportar como Excel (.xlsx)
- ☐ Relatório de ocorrências: todas as RECUSADAS + RETIDAS + ITEM FALTANDO do período
- ☐ Relatório por motorista: histórico individual exportável
- ☐ Relatório por empresa cliente: todas as NFs com status do período

3.8 Critérios de Aceite Adicionais (MVP B)

Funcionalidades

- ☐ **Roteirização:** Calcular rota para um conjunto de 10–20 endereços em menos de 5 segundos
- ☐ **Roteirização:** Botão "Abrir no Maps" abre Google Maps nativo com a parada correta
- ☐ **KPIs:** Dashboard de motoristas exibe dados corretos para o período selecionado
- ☐ **Financeiro:** Cálculo de rentabilidade funciona quando custo e tarifa estão configurados
- ☐ **Gráficos:** Renderizam em menos de 2 segundos mesmo com 30 dias de dados
- ☐ **Filtros avançados:** Cliente filtra por cidade + período + status simultaneamente sem erro
- ☐ **Exportação:** Excel gerado abre corretamente no Excel e Google Sheets

Performance

- ☐ Google Maps API não ultrapassa limite gratuito durante o piloto (monitorado)
- ☐ Dashboard com gráficos carrega em < 4 segundos com 3 meses de histórico

Custo de Infra Adicional (MVP B)

Serviço adicional	Custo estimado/mês
Google Maps Directions API	R\$ 50–150
Google Maps Distance Matrix (financeiro)	R\$ 30–80
Supabase Pro (mais armazenamento histórico)	+R\$ 50
Total adicional	R\$ 130–280/mês

Mensalidade do MVP B: R\$ 300–800/mês cobre esses custos.

4. FORA DO ESCOPO (v2/v3)

Tudo listado abaixo **não será entregue nas Propostas A ou B**. Qualquer solicitação durante o desenvolvimento que se enquadre aqui será tratada como change request para proposta separada.

Funcionalidade	Proposta	Justificativa
Roteirização automática via GPS	B	Requer Google Maps API + UX complexa
KPIs de desempenho dos motoristas	B	Precisa de histórico suficiente
Controle financeiro / rentabilidade	B	Módulo separado, complexidade alta
Gráficos e dashboards avançados	B	Depende dos KPIs
Notificações WhatsApp automáticas	v2	Sem demanda declarada pelo cliente
Notificações push (browser)	v2	Sem demanda declarada
Emissão de CT-e / NF-e	v3	Requer homologação prefeitura
Integração com ERP Interativo	v3	Explicitamente descartado no MVP
App nativo iOS / Android	v2	PWA resolve a necessidade
Relatório exportável (PDF/Excel)	v2	Não solicitado no MVP
Múltiplos coordenadores com níveis distintos	v2	MVP tem gerência única
Histórico de mais de 12 meses	v2	Definir política de retenção

Regra de ouro: Se durante o desenvolvimento surgir uma feature nova, o time avalia:

- Está dentro do escopo A? → Faz
- É um bug da feature contratada? → Faz sem custo adicional
- É feature nova? → Registra como change request, propõe custo separado

5. STACK TÉCNICO DEFINITIVO

4.1 Visão Geral

ALIANÇA LOG	
Frontend	Next.js 14 + TypeScript + Tailwind
Hosting	Vercel (edge, auto-deploy via Git)
Backend	Next.js API Routes (serverless)
Banco	Supabase (PostgreSQL + RLS)
Auth	Supabase Auth (JWT, 3 roles)
Real-time	Supabase Realtime (WebSockets)

Storage	Supabase Storage (fotos canhotos)
Offline	Service Worker + IndexedDB (PWA)
CI/CD	GitHub → Vercel (auto-deploy)

4.2 Decisões Técnicas Detalhadas

Frontend — Next.js 14 App Router

Por quê: Stack principal da UzzAI — equipe já domina, zero curva de aprendizado. App Router permite Server Components (performance) e Client Components (interatividade) no mesmo projeto.

Principais libs:

- `@supabase/supabase-js` — client SDK do Supabase
- `@supabase/auth-helpers-nextjs` — helpers de auth para Next.js
- `tailwindcss` — estilização (sistema de design da UzzAI)
- `xlsx` — parser de Excel no servidor
- `react-hook-form` — formulários (login, upload, status)
- `swr` ou `react-query` — cache e refetch inteligente
- `workbox` — Service Worker para offline

Backend — Next.js API Routes + Supabase

As rotas de API ficam em `app/api/` (serverless functions no Vercel). Lógica:

- `/api/entregas` — CRUD de entregas
- `/api/canhotos` — registro + upload de foto
- `/api/import` — parse e criação em lote via Excel
- `/api/sync` — recebe batch de registros offline dos motoristas

Row Level Security (RLS) no Supabase:

```
-- Motorista vê apenas suas próprias entregas
CREATE POLICY "motorista_own_entregas" ON entregas
  FOR SELECT USING (motorista_id = auth.uid());

-- Cliente vê apenas entregas da sua empresa
CREATE POLICY "cliente_own_notas" ON notas_fiscais
  FOR SELECT USING (empresa_cliente_id = (
    SELECT empresa_id FROM usuarios WHERE id = auth.uid()
  ));
```

Storage de Fotos — Supabase Storage

Por quê: Integrado ao Supabase — sem configuração extra de S3/R2.

Fluxo de upload:

1. Motorista tira foto → comprime no frontend (max 800px, ~200KB)
2. Upload para bucket privado `canhotos` no Supabase Storage
3. Supabase gera URL pública com token temporário (24h)
4. URL salva no banco junto com o registro do canhoto
5. Gerência e cliente acessam a foto via URL — sem download do servidor

Custo estimado: ~100 fotos/dia × 200KB = 20MB/dia = 600MB/mês → **< R\$ 5/mês no plano Pro**

Real-time — Supabase Realtime

Por quê: Nativo ao Supabase — uma linha de código para escutar mudanças no banco.


```
// Dashboard da gerência – escuta mudanças na tabela canhotos
const channel = supabase
  .channel('canhotos')
  .on('postgres_changes', { event: 'INSERT', schema: 'public', table: 'canhotos' },
    (payload) => atualizaDashboard(payload.new))
  .subscribe()
```

Quando um motorista registra um canhoto:

1. Dado é inserido no banco (incluindo foto URL)
2. Supabase Realtime emite evento via WebSocket
3. Dashboard da gerência atualiza em < 1 segundo
4. Portal do cliente atualiza em < 1 segundo

Sem polling, sem refresh manual.

Offline — PWA com Service Worker

Contexto: Motoristas em Gramado, Canela, Petrópolis têm sinal fraco ou inexistente.

Implementação:

```
Motorista sem internet:
1. Abre app → Service Worker serve do cache (instalado no primeiro acesso)
2. Lista de entregas carrega do IndexedDB (atualizado na última vez com sinal)
3. Registra canhoto → foto e dados vão para IndexedDB com flag pendente
4. Sinal volta → background sync dispara → envia todos os registros pendentes
5. Supabase recebe, salva, emite evento Realtime → gerência vê
```

Libs: `workbox-webpack-plugin` (integrado ao Next.js via `next-pwa`)

CI/CD — GitHub + Vercel

Fluxo:

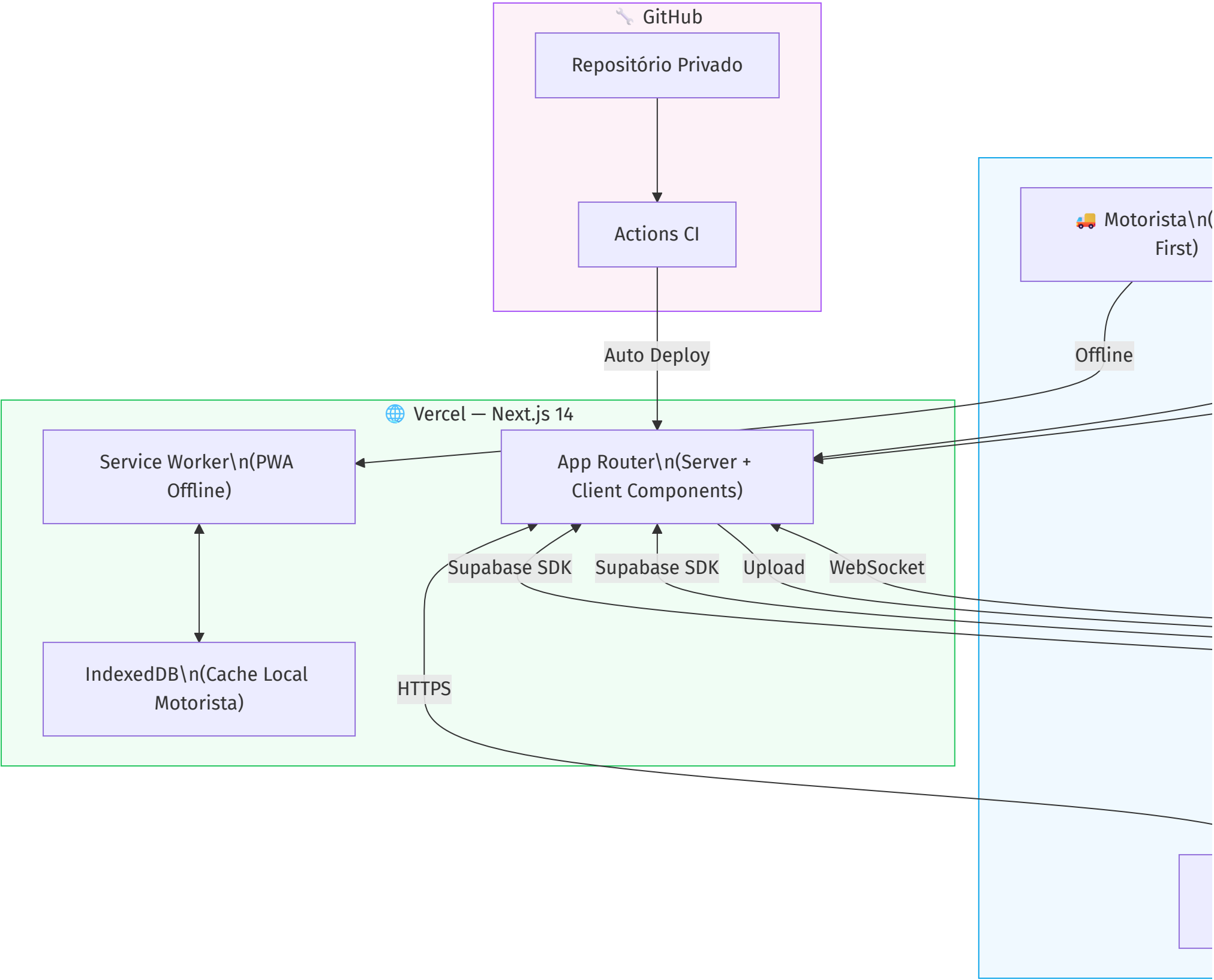
```
Desenvolvedor → git push → GitHub → Vercel detecta push
→ Build Next.js automático → Deploy em ~2 minutos
→ URL de preview para cada PR
→ Merge na main → Deploy em produção automático
```

Environments:

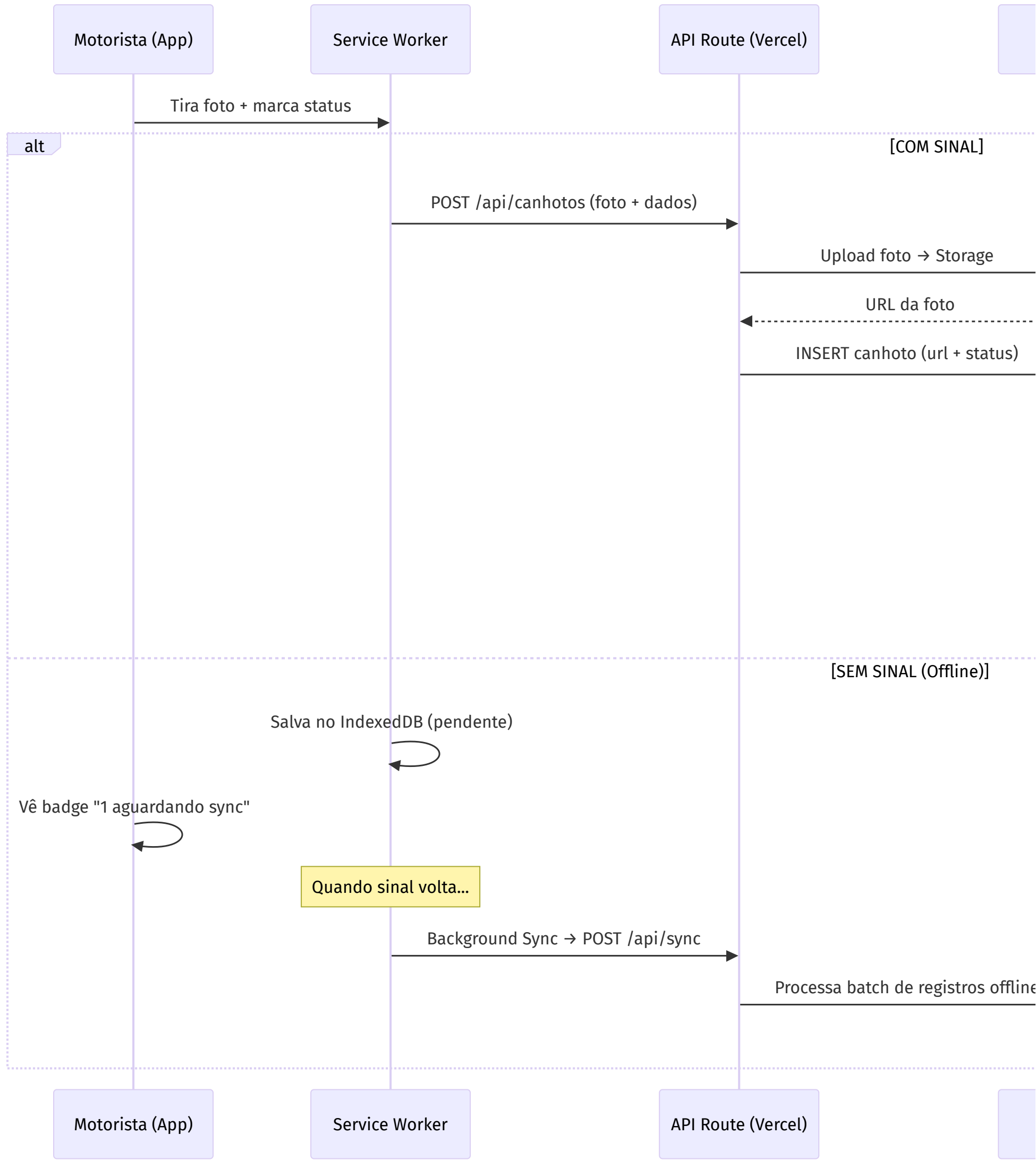
- `main` → produção (`aliancalog.uzzai.com.br`)
- `develop` → staging (`staging.aliancalog.uzzai.com.br`)
- PR → preview automático (`pr-123.aliancalog.uzzai.com.br`)

6. ARQUITETURA DO SISTEMA

5.1 Diagrama Geral



5.2 Fluxo de Dados — Registro de Canhoto



5.3 Estrutura de Pastas do Projeto

```
alianca-log/
├── app/
│   ├── (auth)/
│   │   └── login/      → Tela de login
│   ├── gerencia/
│   │   ├── dashboard/ → Lista de entregas em tempo real
│   │   ├── importar/  → Upload e parse do Excel
│   │   └── cadastros/  → Gestão de motoristas e clientes
│   ├── motorista/
│   │   ├── entregas/  → Lista do dia
│   │   └── canhoto/[id]/ → Tela de registro de canhoto
│   ├── cliente/
│   │   └── notas/     → Portal read-only
```

```

└─ api/
    │   └─ entregas/           → CRUD entregas
    │   └─ canhotos/         → Upload + registro
    │   └─ import/           → Parser Excel
    └─ sync/                 → Sync offline batch
└─ components/
    │   └─ ui/                → Botões, cards, modais
    │   └─ gerencia/         → Componentes do dashboard
    │   └─ motorista/       → Componentes do app field
    └─ cliente/             → Componentes do portal
└─ lib/
    │   └─ supabase/         → Client + Server Supabase
    │   └─ excel-parser/     → Parser .xlsx
    └─ offline/             → IndexedDB helpers
└─ public/
    │   └─ manifest.json     → PWA manifest
    └─ sw.js                → Service Worker
└─ supabase/
    └─ migrations/          → SQL migrations

```

7. BANCO DE DADOS — SCHEMA

6.1 Tabelas Principais

```
-- Empresas clientes (embarcadores: Leite Travizão, Aurora, etc.)
CREATE TABLE empresas_clientes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    nome TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Usuários do sistema (todos os 3 tipos)
CREATE TABLE usuarios (
    id UUID PRIMARY KEY REFERENCES auth.users ON DELETE CASCADE,
    nome TEXT NOT NULL,
    role TEXT NOT NULL CHECK (role IN ('gerencia', 'motorista', 'cliente_final')),
    empresa_id UUID REFERENCES empresas_clientes, -- apenas para cliente_final
    ativo BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Veículos
CREATE TABLE veiculos (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    placa TEXT NOT NULL UNIQUE,
    tipo TEXT,
    ativo BOOLEAN DEFAULT TRUE
);

-- Motoristas (extensão do usuário)
CREATE TABLE motoristas (
    id UUID PRIMARY KEY REFERENCES usuarios(id),
    veiculo_id UUID REFERENCES veiculos,
    telefone TEXT
);

-- Notas Fiscais (dados das entregas)
CREATE TABLE notas_fiscais (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    numero_nf TEXT NOT NULL,
    empresa_cliente_id UUID NOT NULL REFERENCES empresas_clientes,
    destinatario_nome TEXT NOT NULL,
```

```

destinatario_endereco TEXT NOT NULL,
motorista_id UUID REFERENCES motoristas,
data_entrega DATE NOT NULL DEFAULT CURRENT_DATE,
status TEXT NOT NULL DEFAULT 'pendente'
    CHECK (status IN ('pendente', 'aceita', 'recusada', 'retida', 'item_faltando')),
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Canhotos (registro do motorista em campo)
CREATE TABLE canhotos (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    nota_fiscal_id UUID NOT NULL REFERENCES notas_fiscais ON DELETE CASCADE,
    motorista_id UUID NOT NULL REFERENCES motoristas,
    foto_url TEXT,                                -- URL no Supabase Storage
    status TEXT NOT NULL
    CHECK (status IN ('aceita', 'recusada', 'retida', 'item_faltando')),
    item_faltante TEXT,                            -- preenchido quando status = item_faltando
    registrado_em TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    sincronizado BOOLEAN DEFAULT TRUE,             -- FALSE = veio do offline sync
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Índices para performance
CREATE INDEX idx_nf_data ON notas_fiscais(data_entrega);
CREATE INDEX idx_nf_empresa ON notas_fiscais(empresa_cliente_id);
CREATE INDEX idx_nf_motorista ON notas_fiscais(motorista_id);
CREATE INDEX idx_canhotos_nf ON canhotos(nota_fiscal_id);

```

6.2 Row Level Security (RLS)

```

-- Habilitar RLS em todas as tabelas
ALTER TABLE notas_fiscais ENABLE ROW LEVEL SECURITY;
ALTER TABLE canhotos ENABLE ROW LEVEL SECURITY;

-- GERÊNCIA: vê tudo
CREATE POLICY "gerencia_all" ON notas_fiscais
    FOR ALL USING (
        (SELECT role FROM usuarios WHERE id = auth.uid()) = 'gerencia'
    );

-- MOTORISTA: vê apenas suas entregas do dia
CREATE POLICY "motorista_own" ON notas_fiscais
    FOR SELECT USING (
        motorista_id = auth.uid()
        AND data_entrega = CURRENT_DATE
    );

-- MOTORISTA: pode inserir canhotos das suas entregas
CREATE POLICY "motorista_insert_canhoto" ON canhotos
    FOR INSERT WITH CHECK (motorista_id = auth.uid());

-- CLIENTE FINAL: vê apenas NFs da sua empresa
CREATE POLICY "cliente_own_empresa" ON notas_fiscais
    FOR SELECT USING (
        empresa_cliente_id = (
            SELECT empresa_id FROM usuarios WHERE id = auth.uid()
        )
    );

```

8. TELAS E FLUXOS POR PERFIL

7.1 Gerência — Wireframe Mental

ALIANÇA LOG – Gerência

[Importar Excel] [+]

Hoje: 52 entregas

✓

38 aceitas

✗

4 recusadas

⚠

3 retidas

📦

2 c/ pendência

🔄

5 em aberto

Filtrar: [Todos ▼] [Motorista ▼] [Empresa ▼]

NF 12345 • Leite Travizão • João (motorista)
Endereço: Rua X, Gramado •

✓

 ACEITA • 09:32

NF 12346 • Aurora • Carlos (motorista)
Endereço: Av Y, Canela •

🔄

 PENDENTE

NF 12347 • Leite Travizão • João
Endereço: Rua Z, Caxias •

📦

 ITEM FALTANDO • 11:15
→ "Faltou o produto X (2 caixas)"

Ao clicar em qualquer entrega: abre modal com foto do canhoto + todos os detalhes.

7.2 Motorista — Wireframe Mental

Minhas Entregas

8 de 16 concluídas

✓

 NF 12345

✓

 NF 12346

🔄

 NF 12347

🔄

 NF 12348

🔄

 NF 12349

🔍

 Buscar NF ...

→

Registrar Canhoto

NF: 12345

Leite Travizão

Rua X, Gramado

[TIRAR FOTO]

[preview aqui]

[

✓

 ACEITA]

[

✗

 RECUSADA]

[

⚠

 RETIDA]

[

📦

 FALTOU ITEM]

[CONFIRMAR]

Paleta mobile: botões grandes (min 48px), fontes grandes (16px+), mínimo de texto.

7.3 Cliente Final — Wireframe Mental

ALIANÇA LOG – Leite Travizão

[🔍 Filtrar]

Status: [Todas ▼]

Período: [Hoje ▼]

NF 12345 • Rua X, Gramado

✓

 ENTREGUE • 09:32 • [Ver canhoto]

NF 12347 • Rua Z, Caxias

📦

 PENDÊNCIA • "Faltou produto X" • [Ver foto]

NF 12350 • Av W, Canela

🔄

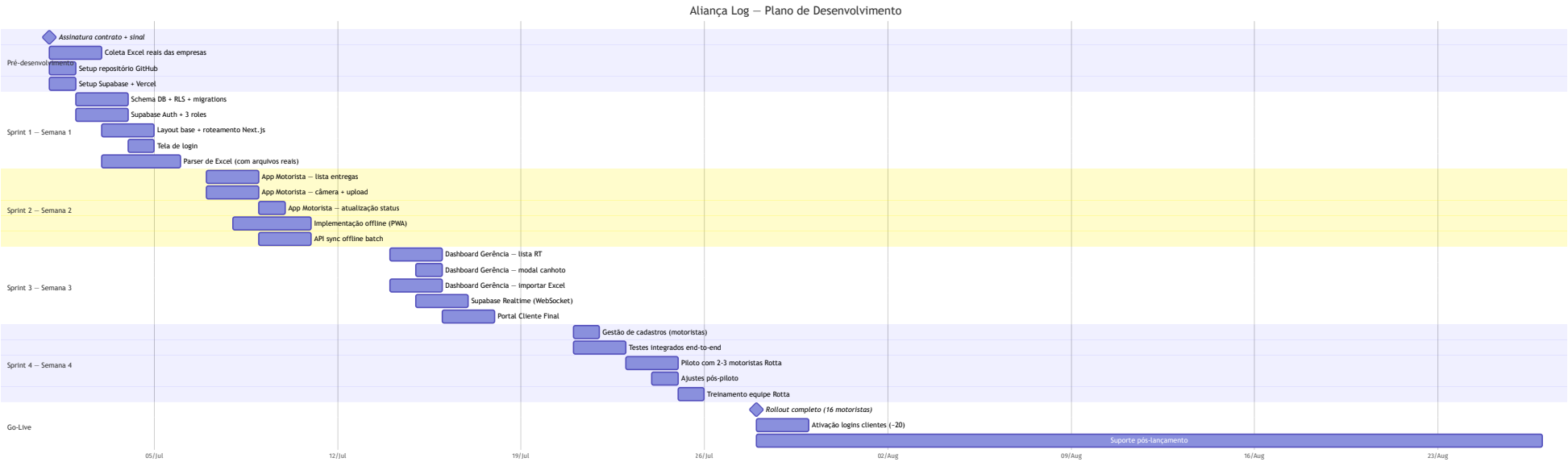
 EM ROTA (aguardando entrega)

9. PLANO DE SPRINTS E TIMELINE

8.1 Premissas

- **Equipe:** Vitor Pirolli (lead), Luis Boff (backend/infra) — ~4h/dia cada
- **Início:** Assim que contrato assinado e 50% do sinal recebido
- **Meta:** Sistema em produção com piloto de 2–3 motoristas em 3 semanas, rollout completo em 4 semanas

8.2 Cronograma Detalhado



8.3 Marcos (Milestones)

Marco	Data	Critério
M0 — Contrato assinado	Após aprovação da proposta	Sinal recebido + repositório criado
M1 — Sprint 1 entregue	Dia 5	Auth funcionando, Excel importado com dados reais
M2 — Sprint 2 entregue	Dia 10	Motorista registra canhoto, offline funciona
M3 — Sprint 3 entregue	Dia 15	Gerência vê em tempo real, cliente acessa portal
M4 — Piloto (2–3 motoristas)	Dia 18	Primeira entrega real registrada no sistema
M5 — Ajustes pós-piloto	Dia 21	Feedbacks do piloto incorporados
M6 — Treinamento equipe	Dia 22	Todos os motoristas treinados
M7 — Go-Live	Dia 25–28	100% dos motoristas no sistema, clientes com acesso

10. EQUIPE E PAPÉIS

9.1 Time Alocado — Aliança Log

Membro	Role	Dedicação	Responsabilidades
Vitor Pirolli	Dev Lead + Account Manager	4–6h/dia	Frontend (motorista + cliente), comunicação com Matheus, testes de usuário, treinamento, piloto
Luis Fernando Boff	Dev Backend + Infra	3–4h/dia	Supabase schema + RLS, API routes, real-time, offline sync, storage, deploy Vercel
Pedro Vitor Pagliarin	CEO + Apoio Técnico	1–2h/dia	Arquitetura, code review, contratos, decisões de escopo, escalção de riscos

9.2 Regras de Colaboração

- **Branch strategy:** **main** (produção) + **develop** (staging) + branches por feature
- **Commits:** ao fim de cada bloco de trabalho (mín. 1x/dia)
- **PRs:** sempre via Pull Request com code review (Vitor ↔ Luis)
- **Comunicação:** grupo exclusivo no WhatsApp (Vitor + Luis + PV + Matheus)

- **Status semanal:** atualização de progresso toda sexta às 17h para Matheus
- **Reuniões:** apenas quando necessário — preferência por assíncrono

9.3 Responsabilidades do Cliente (Rotta)

Responsabilidade	Responsável (Rotta)	Prazo
Fornecer 2–3 arquivos Excel reais das empresas	Matheus Rotta	Dia 1
Listar todos os 16 motoristas (nome + e-mail)	Matheus Rotta	Dia 1
Listar todas as ~20 empresas clientes (nome + e-mail acesso)	Matheus Rotta	Dia 1
Participar do piloto na Semana 4 (2–3 motoristas)	Matheus Rotta	Semana 4
Coordenar rollout com os motoristas	Matheus Rotta	Semana 4
Feedback do piloto em até 24h	Matheus Rotta	Semana 4

11. SERVIÇOS UZZAI INCLUÍDOS

10.1 Desenvolvimento

- ✓ Desenvolvimento completo do MVP A conforme escopo deste documento
- ✓ Código-fonte versionado no GitHub (repositório privado)
- ✓ Deploy automático via Vercel com URL de produção
- ✓ Configuração completa de ambientes (Supabase, Vercel, Storage)
- ✓ Criação de todos os logins (motoristas + clientes finais + gerência)

10.2 Jurídico

- ✓ **Contrato de Desenvolvimento de Software** — cobre:
 - Escopo exato (este documento é anexo)
 - Cronograma com marcos
 - Forma de pagamento (parcelas)
 - Propriedade intelectual (quem fica com o código)
 - Cláusula de confidencialidade
 - SLA de suporte
 - Condições de rescisão
 - Multa por inadimplência
- ✓ **NDA (Non-Disclosure Agreement)** — proteção mútua dos dados do cliente e da tecnologia da UzzAI
- ✓ **Política de Privacidade e Termos de Uso do sistema** — LGPD compliant, publicada no sistema
- ✓ **Cláusulas de SLA** com tempos de resposta definidos por criticidade

Nota: A UzzAI possui suporte jurídico interno (constituição CNPJ ativo desde 12/2025). Contratos são revisados internamente e assinados digitalmente.

10.3 Implantação e Treinamento

- ✓ Configuração do domínio (ex.: aliancalog.uzzai.com.br ou domínio próprio do cliente)
- ✓ Configuração de SSL (HTTPS automático via Vercel)
- ✓ Treinamento dos coordenadores (1 sessão de 1h — presencial ou video-call)
- ✓ Material de apoio para motoristas: guia de 1 página (como registrar canhoto — passo a passo com screenshots)
- ✓ Sessão de ativação dos ~20 clientes finais (configuração dos logins)

10.4 Suporte Pós-Lançamento

- ✓ **30 dias de suporte prioritário** incluso no setup (bugs + ajustes urgentes)
- ✓ **60 dias de correção de bugs sem custo** (erros na funcionalidade entregue)
- ✓ **Suporte contínuo** via mensalidade (após os 30 dias iniciais):

- Dúvidas e orientações
- Pequenos ajustes de interface
- Monitoramento de uptime
- Atualizações de segurança das dependências

12. CRITÉRIOS DE ACEITE (DoD)

A entrega é considerada concluída quando **todos** os critérios abaixo forem atendidos.

11.1 Funcionalidades Core

- ☐ **Auth:** Todos os 3 roles conseguem fazer login, são redirecionados corretamente e não conseguem acessar área do outro role
- ☐ **Importação Excel:** Arquivo Excel real da Leite Travizão ou Aurora importa sem erros, gerando as entregas corretamente
- ☐ **Motorista — lista:** Motorista logado vê exatamente suas entregas do dia atribuídas
- ☐ **Motorista — câmera:** Foto tirada no celular é aceita e enviada com sucesso
- ☐ **Motorista — status:** Os 4 status funcionam; "Item Faltando" exige campo texto preenchido
- ☐ **Offline:** Com o celular em modo avião, motorista consegue registrar canhoto; ao voltar o sinal, dado aparece no dashboard da gerência
- ☐ **Gerência — real-time:** Quando motorista registra, dashboard atualiza em menos de 3 segundos sem refresh manual
- ☐ **Gerência — foto:** Clicando na entrega, a foto do canhoto aparece em boa resolução
- ☐ **Gerência — importar:** Upload de Excel gera entregas e permite atribuição de motoristas
- ☐ **Cliente Final:** Login do cliente vê apenas as NFs da sua empresa; status e foto aparecem quando disponíveis

11.2 Performance

- ☐ Carregamento inicial do app: < 3 segundos em 4G
- ☐ Upload de foto: < 5 segundos em 4G (foto comprimida ~200KB)
- ☐ Latência real-time (registro → gerência): < 3 segundos
- ☐ App do motorista funciona em celulares Android com 2GB RAM (Android 9+)

11.3 Segurança

- ☐ Motorista não consegue ver entregas de outro motorista
- ☐ Cliente Final não consegue ver NFs de outra empresa
- ☐ Sem dados sensíveis em localStorage (apenas IndexedDB com dados não-sensíveis)
- ☐ Todas as comunicações via HTTPS
- ☐ Fotos acessíveis apenas com URL assinada (não indexável pelo Google)

11.4 Operação

- ☐ Sistema rodando em produção com URL fixa configurada
- ☐ Todos os logins dos 16 motoristas criados e testados
- ☐ Todos os logins das ~20 empresas clientes criados
- ☐ Treinamento realizado com pelo menos o coordenador principal
- ☐ Material de apoio (guia motorista) entregue

13. CONTRATO E PAGAMENTO

12.1 Valores — Proposta A (Recomendada)

Item	Valor
Setup / Desenvolvimento	R\$ 10.000
Mensalidade (suporte + infra)	R\$ 350/mês

Valores dentro da faixa R\$ 8.000–12.000 e R\$ 200–400/mês declarados na proposta.

12.2 Forma de Pagamento — Opção 1 (Setup)

Marco	%	Valor	Quando
Sinal / Início	50%	R\$ 5.000	Assinatura do contrato
Entrega (Go-Live)	50%	R\$ 5.000	Sistema em produção aprovado pelo cliente
Total Setup		R\$ 10.000	
Mensalidade	100%	R\$ 350/mês	A partir do mês seguinte ao go-live

12.3 Forma de Pagamento — Opção 2 (Contrato 36 meses)

Alternativa para diluir o custo inicial — validada na reunião interna de 17/06.

Marco	Valor	Quando
Sinal inicial	R\$ 3.000	Assinatura do contrato
Entrega (Go-Live)	R\$ 3.000	Sistema em produção aprovado
Mensalidade (36 meses)	R\$ 650/mês	A partir do mês seguinte
Total 3 anos	R\$ 29.400	
App passa para o cliente ao final	✓	Após 36 meses

Opção 2 tem valor total ~3x maior ao longo de 3 anos, mas elimina a barreira do setup alto à vista.

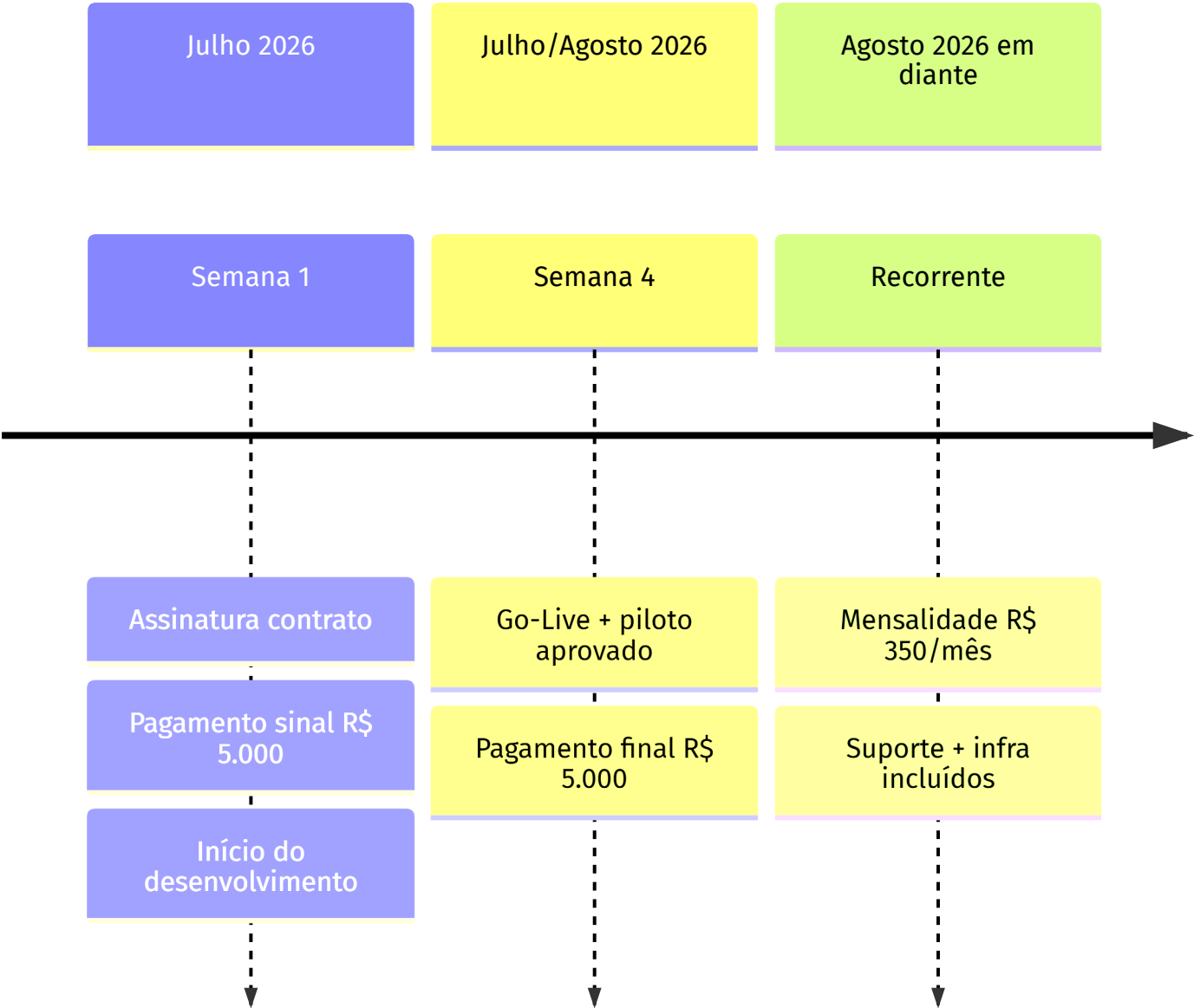
12.4 Propriedade Intelectual

Cenário	Propriedade do Código
Opção 1 (Setup R\$ 10k)	UzzAI mantém a propriedade. Cliente tem licença de uso vitalícia. UzzAI pode usar o produto como base para outros clientes (multi-tenant).
Opção 2 (36 meses)	Ao final dos 36 meses, o código-fonte passa para o cliente.

Esta definição é crítica para a estratégia SaaS da UzzAI. Se optarmos pela Opção 1, protegemos a possibilidade de vender o Aliança Log para outras distribuidoras.

12.5 Cronograma de Pagamento e Marcos (referência contratual)

Aliança Log — Marcos e Pagamentos (Opção 1)



14. GLOSSÁRIO

Termo	Definição
Canhoto	Documento físico ou digital que comprova a entrega (recibo assinado pelo destinatário)
NF / Nota Fiscal	Documento fiscal que acompanha a mercadoria
Embarcador / Cliente Final	Empresa cujas mercadorias são distribuídas (Leite Travizão, Aurora etc.)
Motorista	Entregador da Rotta Logística
Gerência	Coordenadores/proprietários da Rotta
Last-Mile	Último trecho de entrega — do centro de distribuição ao destinatário final
PWA	Progressive Web App — site com capacidades de app (offline, câmera, ícone na tela)
Real-time	Atualização instantânea sem necessidade de recarregar a página
RLS	Row Level Security — segurança a nível de linha no banco de dados
Sync offline	Processo de enviar dados salvos localmente para o servidor quando a internet volta
MVP	Minimum Viable Product — versão mínima funcional para validar o produto
Supabase	Backend-as-a-Service que inclui banco PostgreSQL, auth, storage e real-time
Vercel	Plataforma de deploy para aplicações Next.js
Multi-tenant	Arquitetura onde múltiplos clientes usam a mesma infraestrutura de forma isolada

■ CHECKLIST PRÉ-INÍCIO

Antes de assinar o contrato

- ☐ Luis revisou e aprovou a arquitetura técnica
- ☐ Pedro revisou e aprovou o escopo e valores
- ☐ Matheus escolheu Opção 1 ou Opção 2 de pagamento
- ☐ Definida propriedade intelectual (licença vs. transferência)
- ☐ Contrato assinado digitalmente por ambas as partes

Dia 1 (logo após assinatura)

- ☐ Sinal recebido no Pix/transferência
- ☐ Matheus enviou 2-3 arquivos Excel reais das empresas
- ☐ Matheus enviou lista de motoristas (nome + e-mail)
- ☐ Matheus enviou lista de empresas clientes (nome + e-mail)
- ☐ Repositório GitHub criado e compartilhado com time
- ☐ Supabase project criado (produção + staging)
- ☐ Vercel project configurado com auto-deploy do GitHub

#alianca-log #escopo #tecnico #rotta #logistica #next-js #supabase